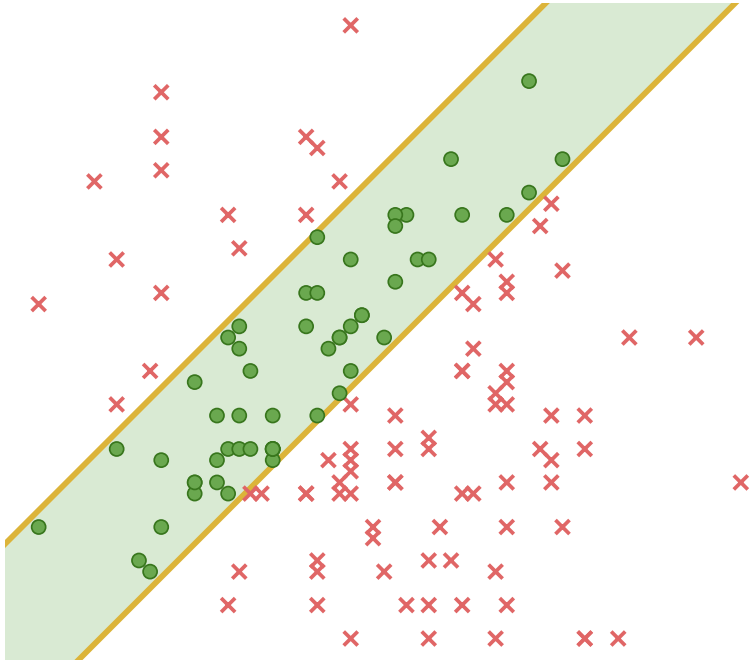
UNIT 02 · CHAPTER 1

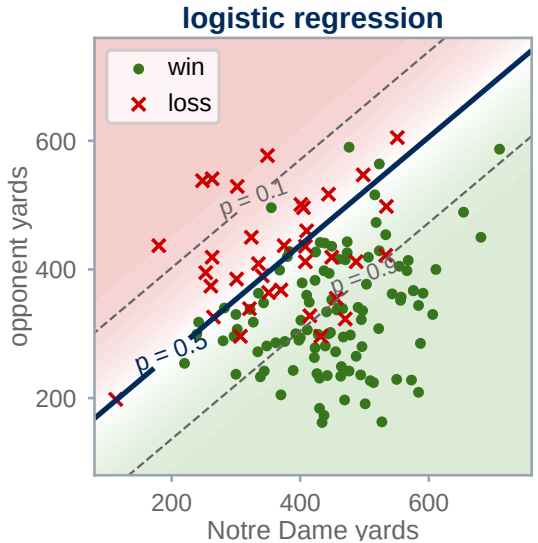
# Feed-Forward Networks

CSE 30124 · Introduction to Artificial Intelligence
William Theisen · University of Notre Dame



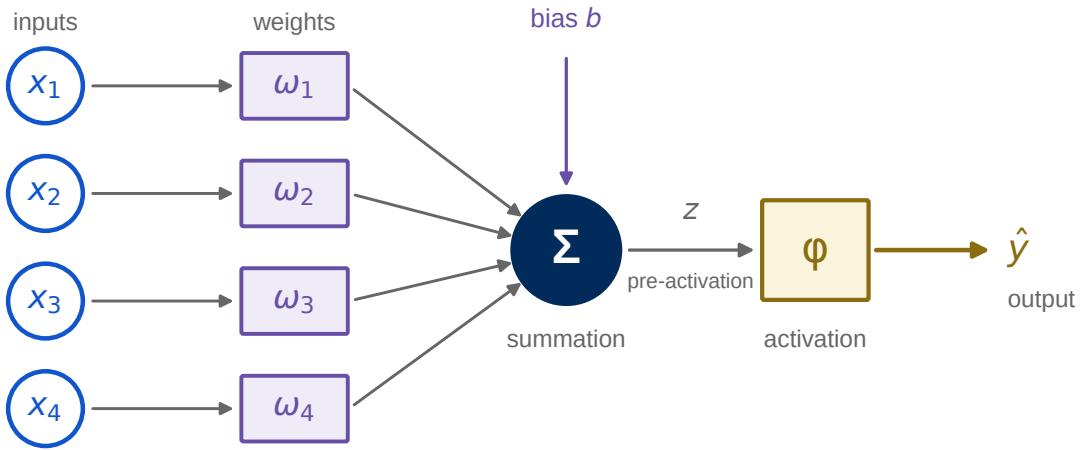
# Predicting Game results

In **Model Evaluation** we asked "Did we win?" and a **logistic regression** answered with a straight line through the two teams' yards, getting **83.7%** of unseen games right.



# Review: Perceptrons

Really, a **logistic regression** is the same thing as a **perceptron**, just with a _____ **activation function**. Generally, we can call this **`unit`** an _____




**Deep learning** is all about how we can use _____

# Artificial Neurons

- _____ **(X):** The information the neuron is receiving
- _____ **(W):** defines how important a given input is for the neuron
- _____ **(b or $\omega_0$):** A small adjustment to the weighted sum of the inputs
  - Think of this as shifting the neurons threshold to make it harder or easier to activate
- **Summation (Σ):** The weighted sum of the inputs and bias
  - represents the neurons "total input", also known as the **pre-activation**

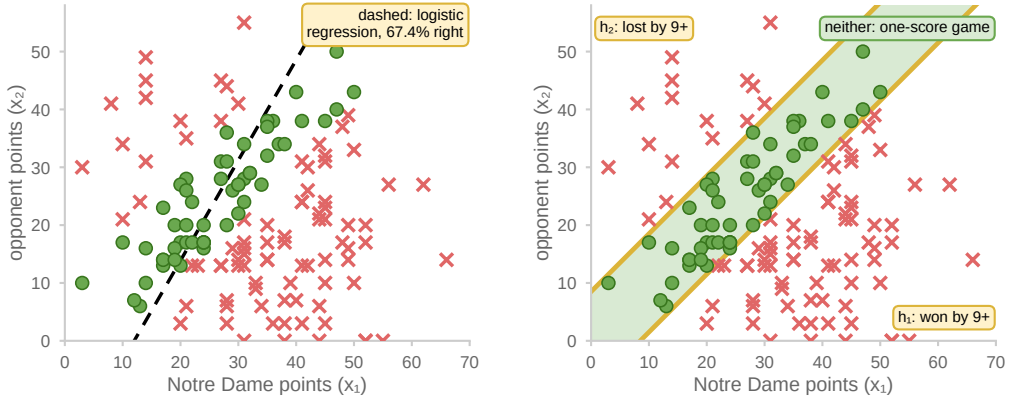
The summation **transforms multiple inputs into one number** that summarizes the information that the neuron cares about.

---

- _____ **(ϕ):** A function applied to the pre-activation of the neuron
- _____ **($\hat{y}$):** The final resulting value from the activation function

# The XOR Problem

What if instead of asking **win** or **loss**, we wanted to **decide whether a game was close or not (within 8 points)**



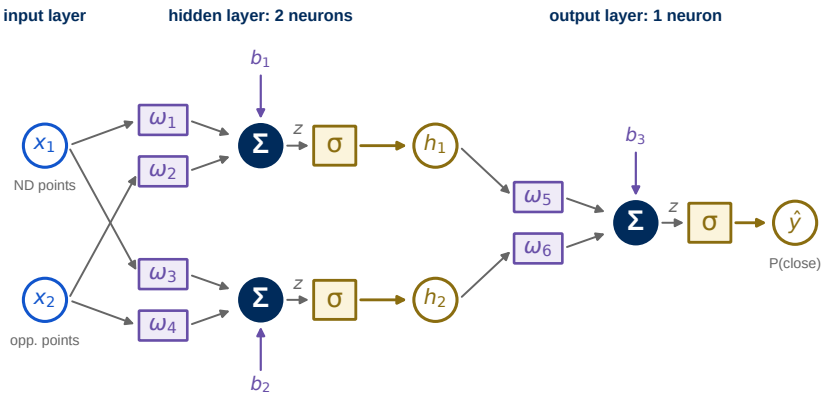




Asking if a game was close or not results in data that is very similar to the **XOR problem** we saw with **perceptrons**, where our **data is not** _____

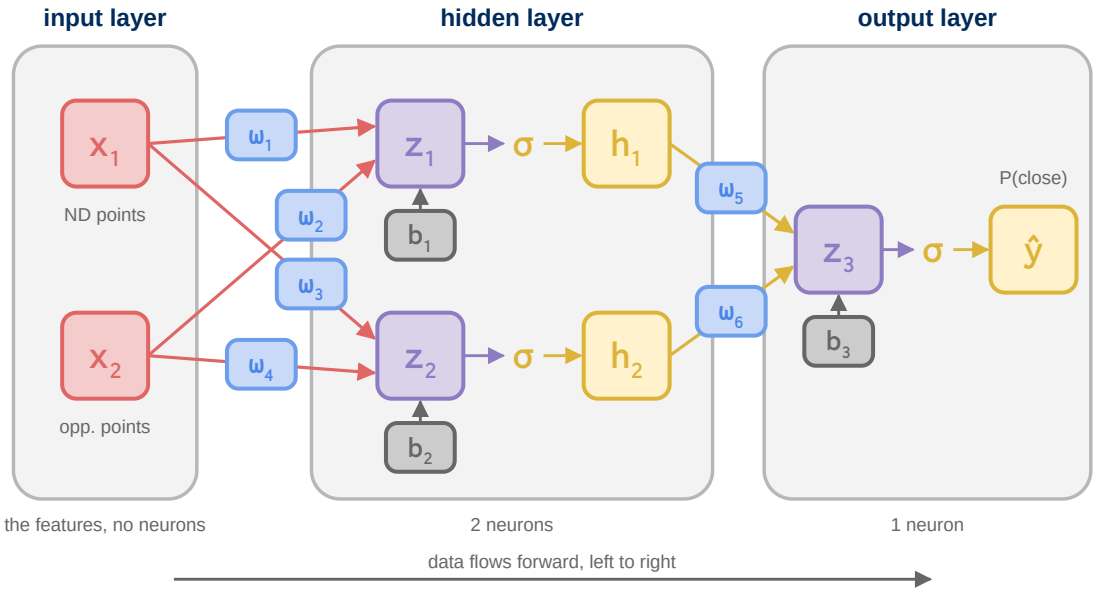
# The XOR Problem

A **neuron** is capable of drawing a single line, but our data requires two lines to separate. We can have two **neurons** each create one of the two lines and use a _____ to **combine their output!**




Stacking **neurons** like this, so some take the outputs of others as their inputs, gives a **multi-layer perceptron (MLP)**, or a **feed-forward neural network (FFN)**

# Neural Networks: Diagram




Combining our three **artificial neurons** gives us a simple **FFN**

# Neural Networks

The organization of neurons in a neural network is called its _____.

Some components that describe an architecture include:

- Number of _____ (**depth**) and number of **neurons** per _____ (**width**)
- **Activation functions** used

# Producing Output from a Neural Network

Producing output with a **neural network** is called "performing the _____"

[0.1] two neurons look at the score

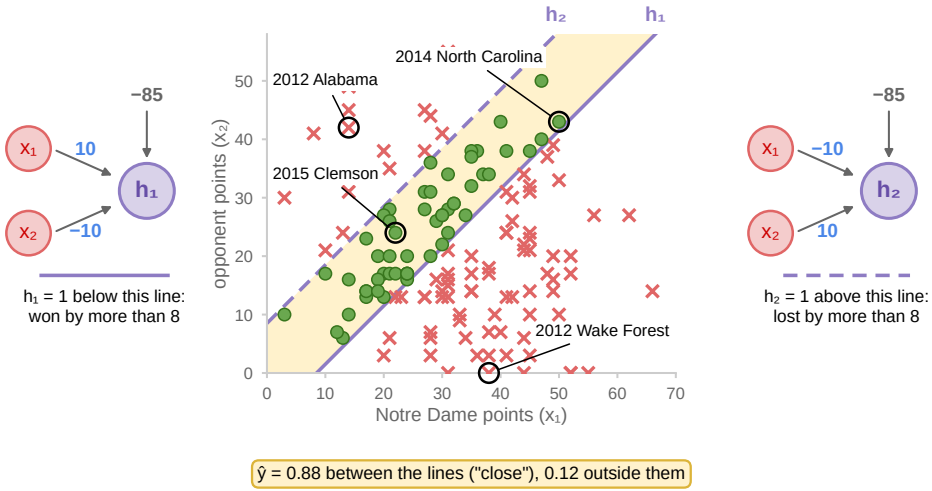
$$h_1 = \sigma(10x_1 - 10x_2 - 85) \qquad h_2 = \sigma(10x_2 - 10x_1 - 85)$$

[0.2] a third combines them

$$\hat{y} = \sigma(2 - 4h_1 - 4h_2)$$

# Decision Regions

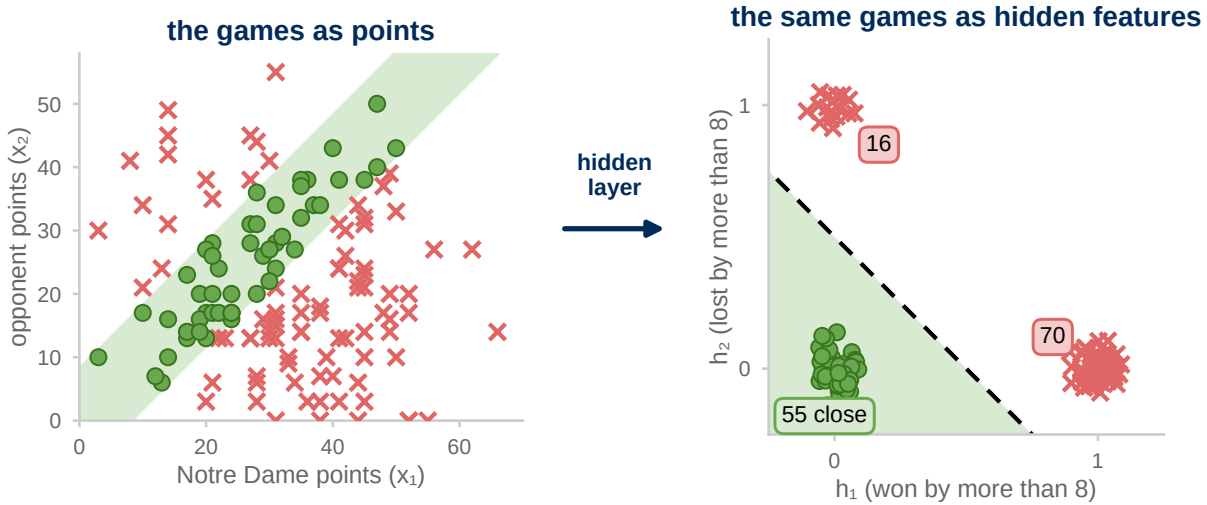
Our first two **neurons** each draw a _____, our third **neuron** combines the two boundaries to create a **decision region**




On all **141** games it gets every one right (**100%**), while no straight line could do better than **88.7%**.

# Hidden Layer Features

**Layers** that don't produce the final output are called _____.




These are what transform our data into _____ points! They create new features for our data that our third **neuron** can separate.

# Weights as Matrices

Much like we stacked our games into a matrix $X$ in Linear Regression, we can stack a layer's neurons into a weight matrix $W$

```
[0.3]
```

one fully connected layer, every game at once

$$Z = XW^\top + b \qquad A = \sigma(Z)$$

# Forward Pass of a Neural Network

```python
import torch

X = torch.tensor([[50., 43.], [38., 0.], [14., 42.]])

W1 = torch.tensor([[10., -10.], [-10., 10.]])
b1 = torch.tensor([-85., -85.])

A1 = torch.sigmoid(X @ W1.T + b1)  # b1 is added to every row

print(X.shape, W1.shape, b1.shape)
print(A1.round(decimals=3))
```

OUTPUT

```
torch.Size([3, 2]) torch.Size([2, 2]) torch.Size([2])
tensor([[0., 0.],
        [1., 0.],
        [0., 1.]])
```

similar to `sklearn` for machine learning, `PyTorch` is the library we will use to implement all of our **neural networks**.

The `@` symbol is how we multiply matrices and `.T` gives us the transpose of a matrix

# Activation Functions

Critically, **activation functions** introduce _____ into the model. This is what allows **neural networks** to learn complex patterns (beyond just linear relationships).

two layers with no activation function

$$Z_2 = (XW_1^T + b_1)W_2^T + b_2$$

multiply out

$$= X(W_2W_1)^T + (b_1W_2^T + b_2)$$

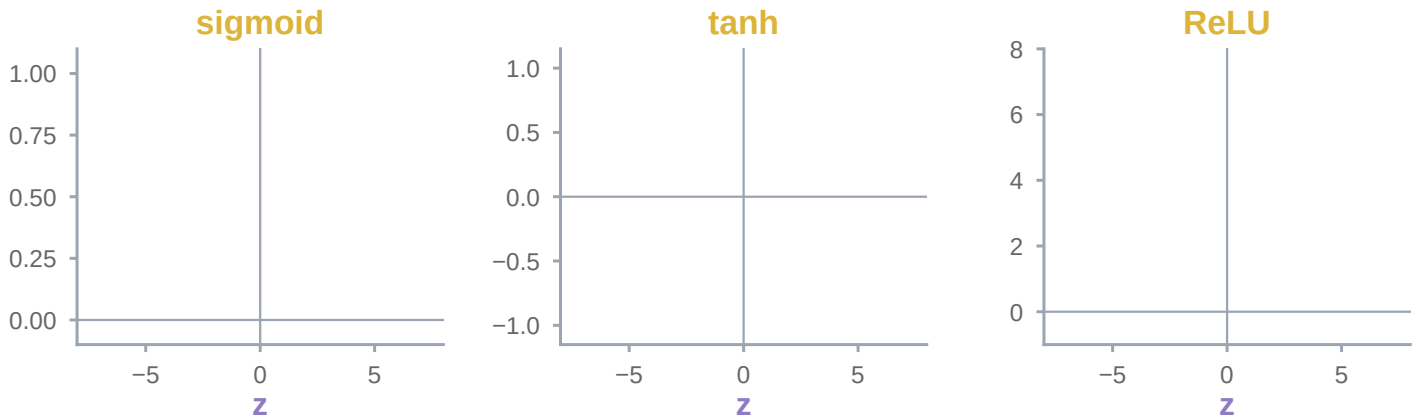
a single layer: its weights are the product of the two, its bias the bracket

$$= XW^T + b$$

If we didn't have **activation functions**, our **neural network** would be no more powerful than just a single **neuron**. However, this power comes at the cost of _____.

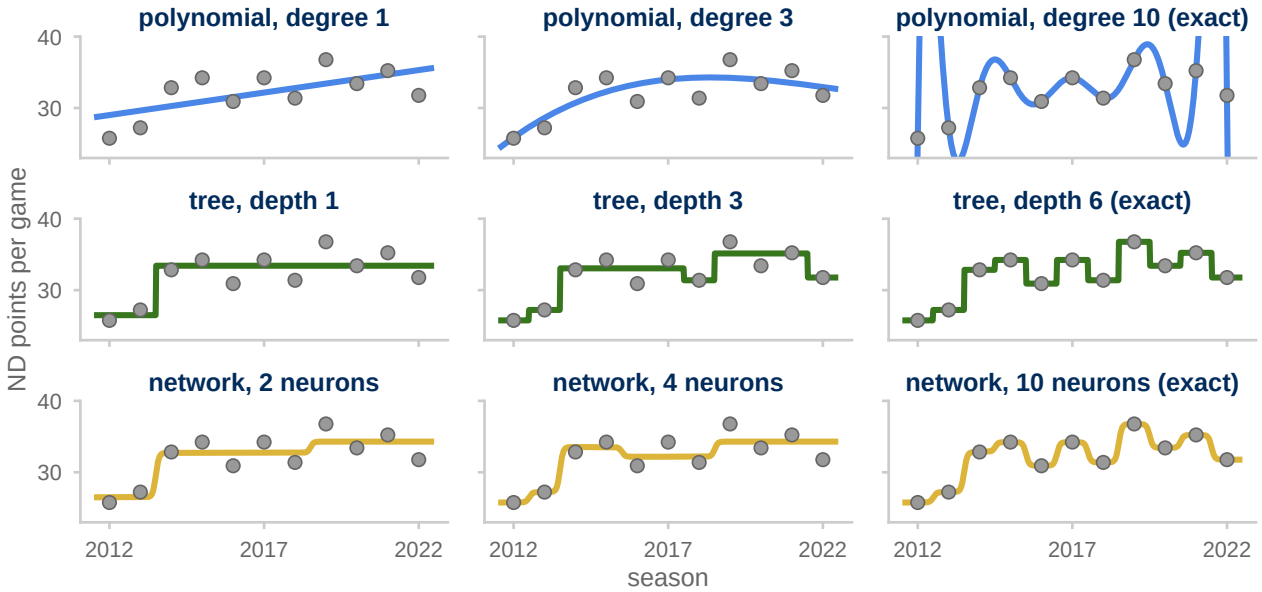
# Activation Functions

For our simple **neural network**, we are using the **sigmoid function** as the **activation function**, but it's not the only choice.




Other common options are the **hyperbolic tangent** and the **ReLU** function

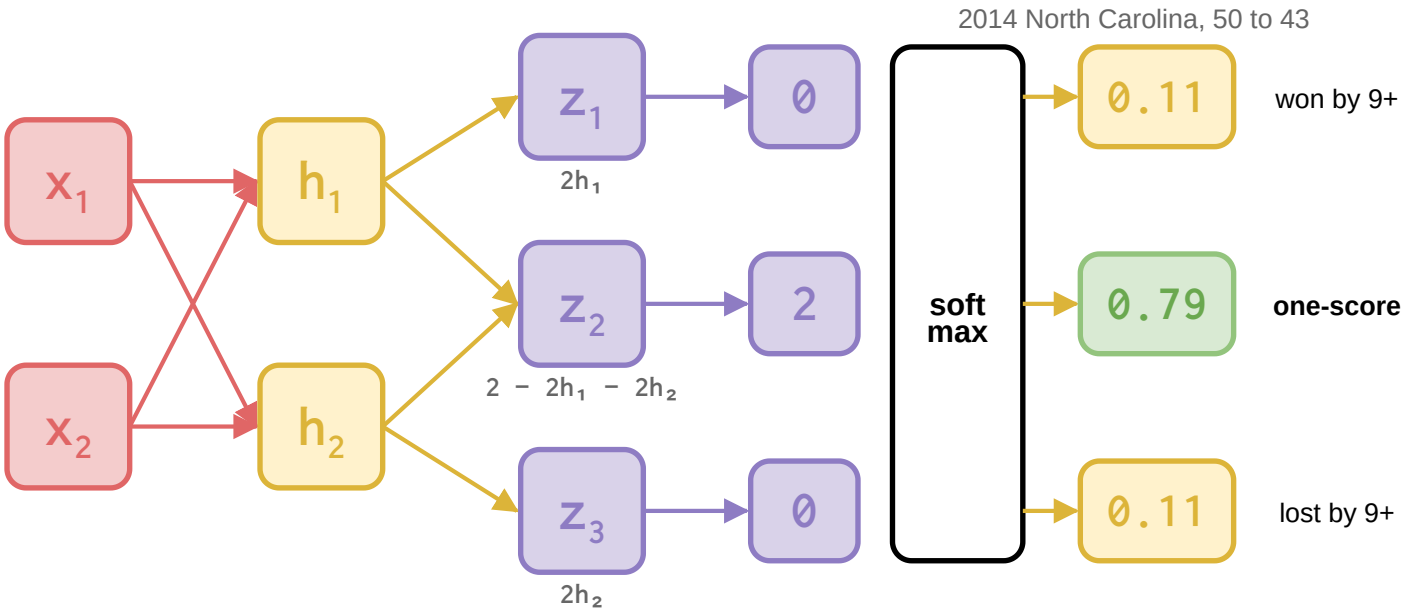
# Universal Approximation Theorem




The **universal approximation theorem** says a **neural network** with a single **hidden layer**, given enough **hidden neurons**, can approximate any continuous function

# Multiclass Classification

Another powerful new ability we gain with **neural networks**, is we are no longer limited to binary outputs, we can use them naturally for **multi-class classification**



# Multi-class Classification

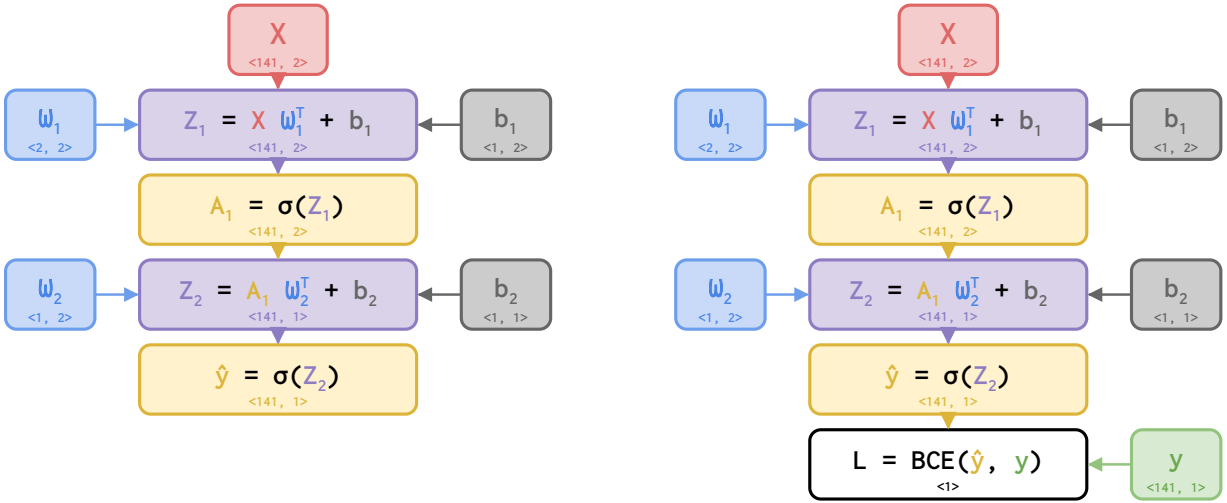
Our three output **scores** aren't **probabilities**. To turn them into probabilities that add up to **1**, we use the _____ function:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

**Softmax** is the usual **activation function** for the _____ of a **classifier**. With only two classes it's the same as the _____.

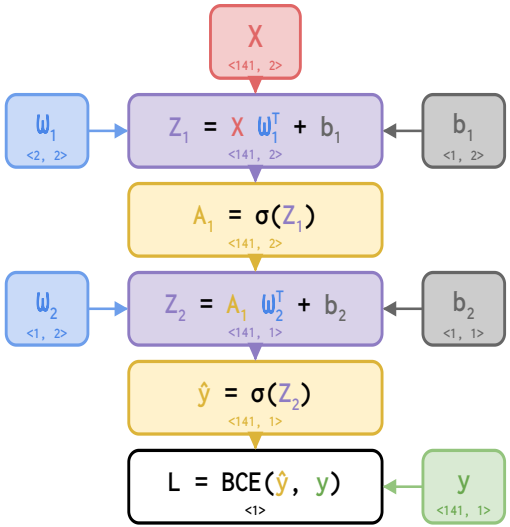
# Representing Neural Networks

Another common way to represent the **architecture** of a **neural network** is as a _____.



# Representing Neural Networks

From the **computation graph**, we can also represent our **neural network** as a single large _____.




This is **critical for training** our **neural networks** (next class)

# An Old Exam Question

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleNetwork(nn.Module):
    def __init__(self):
        super(SimpleNetwork, self).__init__()
        self.fc1 = nn.Linear(2, 32)
        self.fc2 = nn.Linear(32, 16)
        self.fc3 = nn.Linear(16, 1)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = torch.sigmoid(self.fc3(x))
        return x

criterion = nn.BCELoss()
```

# An Old Exam Question

**a.)** Draw the computation graph for the network. (4 pts.)

**b.)** Give the loss of the neural network (L) as a single statement of nested functions. (1 pt.)